Hệ điều hành

Ôn tập giữa kỳ từ kiến thức nền

CSC10007 · Tài liệu tự học bằng tiếng Việt

Tài liệu đi từ cách máy tính chạy một chương trình đến cách hệ điều hành xử lý thiết bị, lưu tập tin và chia CPU. Mỗi chương có thuật ngữ Anh-Việt, ví dụ và câu tự luyện. Phần cuối giải từng câu trong ba đề được cung cấp và hai bài thực hành FAT.

Cách học. Đọc ví dụ trước, sau đó tự làm lại với giấy nháp. Với bài tính, hãy viết quy ước, đơn vị và các bước trung gian. Với câu so sánh, hãy chỉ rõ cơ chế, mục tiêu và điều kiện khiến một cách làm có lợi hơn.

Căn cứ phạm vi. Slide nhập môn dự kiến thi ở tuần 7 sau phần điều phối CPU. Syllabus trình bày kế hoạch cả học phần và ghi giữa kỳ mở tài liệu, nhưng không xác định điểm dừng của nội dung. Ba đề cũ có trọng tâm khác nhau. Chương 1-6 bao phủ các nhóm câu đã gặp. Chương 7 giới thiệu thêm đồng bộ, bộ nhớ và phần mở rộng để dùng khi phạm vi lớp có các mục này. Thông báo của giảng viên là căn cứ cho kỳ thi hiện tại.

Mục lục

01 · Hệ điều hành và phần cứng 2

02 · Ngắt, bảo vệ và I/O 4

03 · HDD, SSD và lập lịch đĩa 6

04 · Tập tin, cấp phát và FAT 9

05 · Tiến trình, luồng và fork 13

06 · Điều phối CPU 16

07 · Đồng bộ và bộ nhớ 19

08 · Lời giải ba đề giữa kỳ 22

09 · Thực hành FAT và tra nhanh 27

01 · Hệ điều hành và phần cứng

Sau chương này, bạn có thể nói CPU, RAM và ổ lưu trữ làm gì, giải thích vai trò của hệ điều hành và phân biệt đa chương với chia sẻ thời gian.

1. CPU, RAM và ổ lưu trữ

Hãy tưởng tượng bạn mở trình soạn thảo và gõ chữ. Tập tin chương trình nằm trên ổ SSD. Hệ điều hành chuẩn bị chương trình để chạy, ánh xạ mã và dữ liệu cần thiết vào không gian địa chỉ của tiến trình. Khi mã được thực thi, CPU lấy lệnh từ bộ nhớ, giải mã lệnh rồi thực hiện. Chữ đang soạn thường nằm trong bộ nhớ làm việc. Lưu tập tin là yêu cầu ghi dữ liệu ra thiết bị lưu trữ.

Thành phần	Vai trò	Ví dụ
CPU (bộ xử lý)	Thực thi lệnh, tính toán và điều khiển	Cộng hai số, rẽ nhánh, đọc bộ nhớ
Thanh ghi (register)	Nơi giữ trạng thái và dữ liệu rất nhanh bên trong CPU	PC giữ vị trí lệnh theo mô hình CPU
RAM (bộ nhớ chính)	Chứa mã và dữ liệu đang được dùng	Dữ liệu đang xử lý, vùng nhớ tiến trình
HDD/SSD (bộ nhớ thứ cấp)	Lưu dữ liệu lâu dài	Tập tin chương trình, tài liệu
Thiết bị I/O	Giao tiếp với bên ngoài hoặc với bộ nhớ thứ cấp	Bàn phím, card mạng, ổ đĩa

RAM thông thường mất dữ liệu khi mất điện. HDD và SSD giữ dữ liệu đã ghi bền vững. Đừng dùng từ “bộ nhớ” một mình trong câu thi nếu sự khác biệt giữa RAM và ổ đĩa quyết định đáp án.

Trong mô hình von Neumann, lệnh và dữ liệu được biểu diễn trong bộ nhớ. Một vòng cơ bản của CPU gồm **fetch** (lấy lệnh), **decode** (giải mã), **execute** (thực thi). Lệnh nhảy có thể đổi địa chỉ tiếp theo. Bộ điều khiển thiết bị cung cấp thanh ghi trạng thái và cơ chế truyền dữ liệu để phần mềm tương tác với phần cứng.

2. Hệ điều hành làm gì?

Hệ điều hành (OS) cung cấp môi trường để chương trình chạy, quản lý tài nguyên và kiểm soát truy cập. **Kernel** là phần lõi có quyền thực thi các thao tác đặc quyền. Trình soạn thảo có thể yêu cầu kernel mở tập tin bằng một giao diện chung. Kernel kiểm tra quyền, tìm metadata của tập tin và phối hợp với driver để đọc dữ liệu khi cần.

Tài nguyên vật lý	Khái niệm OS cung cấp
Thời gian CPU	Tiến trình, luồng và lịch chạy
RAM	Không gian địa chỉ riêng của tiến trình
Ổ lưu trữ	Tập tin, thư mục và đường dẫn
Thiết bị	Giao diện I/O và driver

Cơ chế (mechanism) cho biết hệ thống làm việc đó bằng cách nào. **Chính sách (policy)** quyết định dùng cơ chế khi nào hoặc cho ai. Ví dụ, chuyển ngữ cảnh là cơ chế lưu và nạp trạng thái CPU. Round Robin là chính sách chọn tiến trình theo hàng đợi và lát thời gian.

3. Từ xử lý theo lô đến chia sẻ thời gian

- **Xử lý theo lô (batch processing).** Gom các công việc và xử lý với ít tương tác trong khi chạy. Spooling dùng vùng lưu trữ làm hàng đợi trung gian, chẳng hạn chứa công việc in.
- **Đa chương (multiprogramming).** Duy trì nhiều công việc sẵn sàng. Khi một việc phải đợi I/O, CPU có thể chạy việc khác để giảm thời gian nhàn rỗi.
- **Chia sẻ thời gian (time-sharing).** Chuyển CPU thường xuyên giữa các công việc để người dùng nhận phản hồi sớm. Ngắt đồng hồ cho phép kernel giành lại CPU khi một việc dùng hết lát thời gian.

Ví dụ, A chạy 2 ms rồi đợi đĩa 8 ms. Nếu B đã sẵn sàng, hệ thống đa chương có thể chạy B trong khoảng A đợi. Nếu A và B đều cần CPU lâu và có người đang tương tác, chia lát giúp B sớm được chạy ngay cả khi A chưa đợi I/O. Với công việc theo lô, ít đòi hỏi phản hồi và thường tự nhường CPU khi chờ I/O, mô hình đa chương đơn giản có thể đạt mục tiêu sử dụng CPU với ít lần chuyển ngữ cảnh do chia lát hơn. Kết quả cụ thể còn tùy tải và chi phí chuyển.

4. Máy thật và máy ảo

Trên máy thật, OS điều khiển phần cứng vật lý qua driver. Trong máy ảo, OS khách nhìn thấy phần cứng ảo do hypervisor cung cấp. Máy ảo hỗ trợ cô lập môi trường, chạy nhiều OS trên một máy, sao lưu trạng thái và thử nghiệm. Nó cần chia sẻ CPU, RAM và I/O của máy chủ. Chi phí ảo hóa phụ thuộc thao tác, hypervisor và hỗ trợ phần cứng. Cài OS trực tiếp thường có đường truy cập thiết bị đơn giản hơn, còn việc thử nhiều cấu hình hoặc khôi phục cả môi trường thường thuận tiện hơn với VM.

Tự luyện

Bài 1. Hai chương trình thay phiên đợi đĩa. OS chuyển CPU khi chương trình đang chạy bị chặn. Mục tiêu trực tiếp là gì?

Giải. Giảm thời gian CPU nhàn rỗi. Đây là mục tiêu điển hình của đa chương.

Bài 2. Một chương trình tính vô hạn và không yêu cầu I/O. Cơ chế nào giúp OS vẫn cấp CPU cho trình soạn thảo?

Giải. Bộ định thời phát ngắt. CPU chuyển vào kernel. Kernel có thể lưu trạng thái chương trình đang chạy và cấp CPU cho trình soạn thảo theo chính sách lập lịch.

Nguồn đối chiếu · 1 Ngay 1 intro, slide 10-32 và 33-43. Architecture Tour, slide 3-21. Operating System Concepts 10e, chương 1, 2 và 18.

02 · Ngắt, bảo vệ và I/O

Sau chương này, bạn có thể giải thích ba cách CPU đi vào kernel, so sánh hai cặp polling/interrupt và PIO/DMA, rồi tính ngưỡng hòa vốn khi đề cho chi phí.

1. User mode và kernel mode

User mode giới hạn quyền của mã ứng dụng. **Kernel mode** cho phép chạy lệnh đặc quyền, chẳng hạn điều khiển bộ định thời và thiết lập cơ chế bảo vệ bộ nhớ. CPU và cơ chế quản lý bộ nhớ kiểm tra quyền đối với các thao tác liên quan. Nếu chương trình user truy cập trực tiếp vùng chỉ dành cho kernel, CPU phát ngoại lệ. Kernel xử lý ngoại lệ theo nguyên nhân, chẳng hạn kết thúc chương trình khi truy cập trái phép.

Đường vào kernel	Nguồn sự kiện	Ví dụ
System call	Chương trình chủ động yêu cầu dịch vụ	Đọc tập tin, tạo tiến trình
Exception	Phát sinh đồng bộ với lệnh đang thực thi	Chia cho 0, lỗi trang, truy cập trái phép
Hardware interrupt	Thiết bị hoặc bộ định thời phát tín hiệu	Card mạng có dữ liệu, hết lát thời gian

Tài liệu có thể dùng **trap** với nghĩa khác nhau. Trong bài giảng Architecture Tour, trap là cách có chủ đích để ứng dụng yêu cầu dịch vụ. Trong một số sách hoặc kiến trúc, trap còn được dùng trong nhóm ngoại lệ đồng bộ. Khi trả lời, hãy gọi tên sự kiện cụ thể.

Ví dụ một lần đọc tập tin chưa có trong bộ đệm. Ứng dụng gọi dịch vụ đọc. CPU vào kernel. Kernel có thể chặn tiến trình đang đợi và chạy tiến trình khác. Khi thiết bị báo hoàn tất bằng ngắt, kernel xử lý kết quả và đưa tiến trình đã đợi về ready. Khi được chọn chạy lại, ứng dụng tiếp tục sau lời gọi. CPU cũng có thể vào kernel rồi quay lại **cùng** tiến trình. Vì vậy **đổi chế độ** và **đổi tiến trình** là hai sự kiện riêng.

2. Vì sao cần ngắt?

Khi CPU đang chạy mã user, một thiết bị có thể hoàn thành I/O vào lúc CPU đang làm việc khác. Ngắt đưa quyền điều khiển về bộ xử lý ngắt của kernel để xử lý sự kiện. Ngắt đồng hồ giải quyết một vấn đề riêng của chia sẻ thời gian. Kernel có thể đặt hẹn giờ trước khi giao CPU cho ứng dụng. Khi hết giờ, CPU vào kernel dù ứng dụng chưa tự nhường, giúp scheduler thực thi chính sách có thu hồi quyền chạy.

Trong mô hình cơ bản, CPU lưu trạng thái cần thiết, chuyển quyền và nhảy đến điểm xử lý tương ứng. Kernel thực hiện phần việc của handler, rồi khôi phục ngữ cảnh thích hợp. Việc ngắt không đồng nghĩa rằng một tiến trình mới luôn được chọn.

3. Polling và interrupt trả lời câu hỏi “thiết bị báo bằng cách nào?”

Ví dụ card mạng nhận ít gói. Nếu CPU kiểm tra 1.000 lần mà chỉ thấy vài gói, phần lớn lượt kiểm tra tốn công vô ích. Nếu gói dồn dập, một ngắt riêng cho mỗi gói có thể tốn nhiều thời gian vào handler. Polling theo lô có thể giảm chi phí. Độ trễ của polling định kỳ phụ thuộc chu kỳ kiểm tra. Câu hỏi “cái nào tốt hơn” cần nêu mật độ sự kiện, chi phí mỗi lần và mục tiêu về độ trễ.

Cách	Hoạt động	Tình huống có lợi
Polling (thăm dò)	CPU chủ động kiểm tra thanh ghi trạng thái, theo vòng lặp hoặc theo chu kỳ	Sự kiện rất thường xuyên, có thể gom nhiều mục trong một lượt, hoặc cần kiểm tra trong khoảng chờ rất ngắn
Interrupt (ngắt)	Thiết bị báo CPU khi sự kiện xảy ra	Sự kiện thưa, khó dự đoán và CPU có công việc khác trong lúc đợi

4. PIO và DMA trả lời câu hỏi “ai chuyển dữ liệu?”

Cách	Ai chuyển dữ liệu?	Chi phí đặc trưng
Programmed I/O (PIO)	CPU thực hiện lệnh đọc/ghi dữ liệu qua thanh ghi thiết bị	Ít bước chuẩn bị, chi phí CPU tăng theo lượng dữ liệu
Direct Memory Access (DMA)	Bộ điều khiển DMA hoặc thiết bị truyền dữ liệu giữa thiết bị và RAM sau khi CPU thiết lập	Có chi phí thiết lập và xử lý hoàn tất, thường có lợi khi truyền khối lớn

CPU có thể dùng polling để biết khi nào DMA hoàn tất, hoặc thiết bị có thể báo bằng ngắt. PIO cũng có thể dùng ngắt để biết khi nào dữ liệu sẵn sàng. Hai cặp khái niệm mô tả hai quyết định khác nhau.

Ví dụ. Một byte cần một load 100 chu kỳ và một store 50 chu kỳ. DMA cần bốn load để chuẩn bị và 4.000 chu kỳ xử lý hoàn tất. Với N byte, PIO tốn 150N chu kỳ CPU, DMA tốn 4.400. DMA dùng ít chu kỳ CPU hơn khi N > 4.400/150, tức từ **30 byte**. Nếu load 100 chỉ thực hiện một lần cho toàn bộ gói và mỗi byte chỉ thêm một store, biểu thức thành 100 + 50N. Khi đó hòa vốn ở **86 byte**, và DMA ít hơn từ 87 byte.

Lưu ý khi đọc slide. Slide 52 mô tả một load và một store *mỗi byte*, trong khi biểu thức ở slide 53 tính một load *cho cả gói*. Hai ngưỡng ở trên tương ứng với hai giả thiết. Khi làm bài, lập công thức theo câu chữ của đề và ghi rõ quy ước.

5. Boot và ngắt trong dòng thực thi

Khi khởi động, CPU bắt đầu tại vị trí đã quy định bởi nền tảng. Firmware chuẩn bị phần cứng và chạy đường khởi động thích hợp. Bootloader hoặc firmware nạp và chuyển quyền cho kernel. Tách mã khởi động khỏi phần còn lại giúp hệ thống cập nhật OS và hỗ trợ nhiều cách khởi động. Trong quá trình chạy, bảng điểm vào cho phép CPU tìm handler đúng với loại sự kiện. Ngữ cảnh thường gồm PC, thanh ghi và trạng thái điều khiển cần cho việc tiếp tục.

Tự luyện

Bài 1. Card mạng có một đợt dữ liệu dày, sau đó im lặng lâu. Giải thích vì sao có thể dùng ngắt lúc ít dữ liệu rồi polling theo lô khi có tải cao.

Giải. Khi ít dữ liệu, ngắt tránh kiểm tra vô ích. Khi dữ liệu dày, polling lấy nhiều mục trong một lượt và tránh xử lý ngắt cho từng mục. Đổi chế độ giúp cân bằng chi phí theo tải.

Bài 2. PIO cần 4 chu kỳ CPU mỗi byte. DMA tốn 120 chu kỳ CPU cố định. Hòa vốn ở đâu?

Giải. 4N = 120 nên N = 30 byte. Với 31 byte trở lên, DMA dùng ít chu kỳ CPU hơn trong mô hình này. Số chu kỳ CPU và thời gian truyền thực tế là hai đại lượng riêng.

Nguồn đối chiếu · Architecture Tour, slide 21-59. Operating System Concepts 10e, chương 1, 2 và 12.

03 · HDD, SSD và lập lịch đĩa

Sau chương này, bạn có thể tính quãng đường đầu đọc cho FCFS, SSTF, SCAN và giải thích các cơ chế chính của HDD, SSD và RAID.

1. HDD đọc dữ liệu như thế nào?

HDD có các mặt đĩa quay. Mỗi mặt có các vòng đồng tâm gọi là **track**. Một **sector** là một phần của track theo mô hình hình học. Tập các track cùng bán kính trên nhiều mặt gọi là **cylinder**. Đầu đọc dịch chuyển để tới vùng cần đọc. Hệ điều hành thường làm việc với địa chỉ khối logic (LBA). Bài tập cylinder dùng mô hình đơn giản hóa để tính chi phí dịch đầu đọc.

Thành phần thời gian	Nghĩa
Seek time	Thời gian dịch đầu đọc đến track/cylinder
Rotational latency	Đợi sector quay đến dưới đầu đọc
Transfer time	Thời gian truyền dữ liệu sau khi đến đúng vị trí
Queue/controller delay	Đợi phục vụ hoặc thời gian xử lý của bộ điều khiển, nếu đề tính

Nếu đĩa quay 7.200 vòng/phút thì một vòng mất 60/7.200 s = 8,33 ms. Giả sử vị trí góc phân bố đều, trung bình đợi nửa vòng, tức 4,17 ms. Với seek 5 ms và transfer 0,4 ms, tổng theo ba thành phần là 9,57 ms. Nếu bài chỉ cho cylinder, dùng tổng |vị trí kế − vị trí trước| làm thước đo seek theo giả thiết bài.

2. Thuật toán lập lịch đầu đọc

Thuật toán	Quy tắc chọn	Đặc điểm
FCFS	Theo thứ tự đến	Dễ thực hiện. Chuỗi yêu cầu xa nhau có thể gây nhiều di chuyển.
SSTF	Chọn yêu cầu gần vị trí hiện tại nhất	Có thể để yêu cầu xa chờ lâu. Tính lại khoảng cách sau mỗi lần chọn.
SCAN	Quét theo một chiều tới biên rồi đảo chiều nếu còn yêu cầu	Cần biết chiều ban đầu và biên cylinder.
C-SCAN	Phục vụ theo một chiều. Tới biên thì quay về biên đối diện.	Đo cả quãng quay về khi bài hỏi tổng dịch chuyển vật lý.
LOOK	Đảo chiều tại yêu cầu xa nhất còn lại theo hướng đang đi	Tránh đi đến biên nếu đoạn cuối không có yêu cầu.
C-LOOK	Phục vụ theo một chiều, rồi chuyển tới yêu cầu ở đầu kia	Đo quãng chuyển nếu tính dịch chuyển vật lý.

Cách làm. Ghi vị trí đầu, miền cylinder, chiều quét và quy tắc hòa khoảng cách nếu cần. Viết thứ tự điểm đầu đọc thực sự đi qua, kể cả biên bắt buộc. Lấy trị tuyệt đối mỗi chặng và cộng. Dừng sau yêu cầu cuối nếu đề hỏi quãng đường phục vụ danh sách. Không tự cộng thêm một chuyến về biên sau đó.

Ví dụ. Cylinder 0-9, đầu đọc tại 5, hướng lên, hàng đợi [0, 4, 7, 9]. Mọi yêu cầu đã có sẵn.

Thuật toán	Đường đi	Tổng cylinder
SCAN	5 → 7 → 9 → 4 → 0	2 + 2 + 5 + 4 = 13
FCFS	5 → 0 → 4 → 7 → 9	5 + 4 + 3 + 2 = 14
SSTF	5 → 4 → 7 → 9 → 0	1 + 3 + 2 + 9 = 15

SSTF chọn 4 ngay vì cách 5 một cylinder. Sau đó 7 gần 4 hơn 0. Quyết định tham lam ở bước đầu khiến yêu cầu 0 phải đợi đến cuối. Đây cũng là một đáp án cho câu 9 đề 2025-2026.

3. Tổ chức đĩa và độ tin cậy

Định dạng mức thấp tổ chức sector và thông tin phục vụ đọc/ghi. Phân vùng chia không gian lưu trữ thành các miền. Định dạng logic khởi tạo hệ thống tập tin trên volume. Vùng swap có thể được dùng làm nơi chứa tạm dữ liệu bộ nhớ, theo cơ chế của OS.

RAID	Nguyên lý	Dung lượng hữu ích với n đĩa cùng cỡ S
RAID 0	Striping, chia dữ liệu ra nhiều đĩa	nS, không có dư thừa
RAID 1	Mirroring, ghi các bản sao	S với một cặp hai đĩa
RAID 2-3	Các mức lịch sử. RAID 2 dùng mã sửa lỗi. RAID 3 dùng parity chuyên biệt với striping nhỏ.	Tùy tổ chức
RAID 4	Striping theo khối và một đĩa parity riêng	(n − 1)S
RAID 5	Phân tán parity giữa các đĩa	(n − 1)S
RAID 6	Hai khối parity theo mỗi stripe	(n − 2)S

RAID 5 có thể chịu mất một đĩa, RAID 6 có thể chịu mất hai đĩa trong mô hình thông thường. RAID phục vụ tính sẵn sàng trước sự cố đĩa. Sao lưu còn cần để phục hồi sau xóa nhầm hoặc dữ liệu bị sửa sai.

Parity và Hamming. Với parity chẵn, thêm bit để tổng số bit 1 chẵn. Nếu một bit đổi, phép kiểm tra phát hiện lỗi, nhưng một parity bit không đủ chỉ vị trí. Mã Hamming dùng nhiều nhóm kiểm tra chồng lấn. Chọn vị trí parity 1, 2, 4, 8… và điền dữ liệu vào vị trí còn lại. Mỗi parity kiểm tra các vị trí có bit tương ứng bật trong chỉ số. Cộng các vị trí parity bị sai để ra syndrome. Quy ước đánh số từ trái hay phải phải giữ nhất quán. Ví dụ kiểm tra nhóm 1 và 2 sai, nhóm 4 đúng thì syndrome 1 + 2 = 3. Với giả thiết một bit lỗi, lật bit ở vị trí 3.

Ví dụ mã Hamming (7,4). Đặt dữ liệu 1, 0, 1, 1 vào vị trí 3, 5, 6, 7. Với parity chẵn, các bit ở vị trí 1, 2, 4 lần lượt là 0, 1, 0, nên từ mã theo vị trí 1 đến 7 là 0110011. Giả sử nhận 0110001. Nhóm kiểm tra 2 và 4 cho tổng lẻ. Syndrome 2 + 4 = 6 chỉ vị trí cần lật.

4. SSD và NAND flash

SSD không có đầu đọc quay. NAND tổ chức từ cell lên page, block, plane, die. Thông thường đọc và program theo **page**, còn xóa theo **block** gồm nhiều page. Cell có giới hạn số chu kỳ program/erase, và bộ điều khiển phối hợp nhiều kênh để tăng song song.

FTL (Flash Translation Layer) ánh xạ địa chỉ logic của host đến page vật lý. Khi cập nhật một page logic, SSD có thể ghi dữ liệu mới sang page trống, đổi ánh xạ và đánh dấu page cũ vô hiệu. **Garbage collection** sao chép page còn hợp lệ ra nơi khác, sau đó xóa cả block để thu hồi chỗ trống. **Wear leveling** phân bố ghi/xóa để giảm hao mòn tập trung. **TRIM** báo những khối logic đã được hệ điều hành giải phóng, giúp FTL bỏ qua dữ liệu không còn cần. **Write amplification** là tỷ số lượng ghi vật lý trên lượng ghi logic của host trong một khoảng đo. Nếu host ghi 4 KiB và GC khiến SSD phải ghi thêm 12 KiB, tỷ số là 4 trong mô hình đó.

Ở cấp chip, tR là thời gian đọc page từ mảng cell, tPROG là thời gian program và tBERS là thời gian xóa block. Bộ điều khiển có thể chồng lấp các thao tác qua nhiều kênh, die hoặc plane khi phần cứng hỗ trợ. Các lệnh như Random Data Out đọc một phần dữ liệu từ bộ đệm page của chip, giúp truyền đúng đoạn cần thiết. Tải ghi tuần tự và tải ghi ngẫu nhiên có thể tạo áp lực thu gom rác khác nhau.

Tự luyện

Bài 1. Miền 0-199, đầu đọc tại 50 đi lên, hàng đợi [82, 10, 65]. Tính SCAN và LOOK.

Giải. SCAN đi 50 → 65 → 82 → 199 → 10, dài 15 + 17 + 117 + 189 = **338**. LOOK đi 50 → 65 → 82 → 10, dài 15 + 17 + 72 = **104**.

Bài 2. Vì sao khi xóa một tập tin trên hệ thống tập tin, SSD có thể chưa xóa block NAND ngay?

Giải. OS bỏ liên kết và giải phóng khối logic. NAND xóa theo block lớn. FTL có thể thu hồi block sau khi di chuyển các page còn hợp lệ. TRIM báo địa chỉ logic đã bỏ để hỗ trợ quá trình này.

Nguồn đối chiếu · 3 disk 1, slide 3-43. SSD và ssd_continue. Operating System Concepts 10e, chương 11.

04 · Tập tin, cấp phát và FAT

Sau chương này, bạn có thể phân biệt các cách cấp phát, đọc BPB, lần chuỗi cluster và xác định vùng thay đổi sau thao tác FAT.

1. Tập tin và thư mục

Tập tin (file) là một đối tượng lưu dữ liệu có tên. **Metadata** là thông tin mô tả, ví dụ kích thước, quyền, ngày giờ, vị trí dữ liệu. **Thư mục (directory)** liên kết tên với mục quản lý của tập tin hoặc thư mục con. Trong cây thư mục, đường dẫn tuyệt đối đi từ gốc, đường dẫn tương đối đi từ vị trí hiện tại. Gắn kết (mount) đưa một hệ thống tập tin vào vị trí truy cập trong cấu trúc tên của OS.

Các thao tác thường gặp gồm tạo, mở, đọc, ghi, đổi tên, di chuyển, xóa và đóng. Sau khi mở, tiến trình thường dùng **file descriptor** hoặc handle để yêu cầu thao tác. Kernel lưu thông tin của tập tin đang mở và vị trí đọc/ghi. Đọc **tuần tự** tiếp tục từ vị trí hiện tại. Đọc **trực tiếp** có thể chọn vị trí cần đọc. Truy cập qua chỉ mục giúp tìm vị trí thông qua một bảng hoặc cấu trúc khóa.

OS có thể dùng lớp **VFS (Virtual File System)** để nhiều hệ thống tập tin cùng cung cấp các thao tác chung. Việc ánh xạ tên, kiểm tra quyền, tìm các block và gọi driver nằm ở các lớp khác nhau. Trong Unix, thư mục ánh xạ tên đến số **inode**. Inode giữ metadata và thông tin ánh xạ block. Trong FAT, mục thư mục mang cluster đầu, còn FAT nối các cluster.

2. Ba phương pháp cấp phát

Cấp phát	Cách xác định block	Ưu điểm và chi phí
Liên tục	Block đầu và số block. Block logic i nằm ở đầu + i.	Truy cập ngẫu nhiên nhanh. Tập tin khó mở rộng nếu hết chỗ kề bên. Có phân mảnh ngoại vi.
Liên kết	Block hoặc bảng giữ địa chỉ block kế.	Tận dụng được các block rời. Phải lần chuỗi để tới một block xa. FAT đưa liên kết vào bảng riêng.
Chỉ mục	Một block chỉ mục hoặc cấu trúc nhiều cấp chứa địa chỉ các block dữ liệu.	Tra cứu theo vị trí nhanh hơn lần chuỗi. Cần không gian cho chỉ mục.

Phân mảnh nội vi là phần dư nằm *bên trong* đơn vị đã cấp. Ví dụ, tập tin 2.500 B dùng ba block 1.024 B, dư 572 B trong block cuối. Cả ba phương pháp có thể gặp hiện tượng này khi cấp theo block cố định. **Phân mảnh ngoại vi** là các lỗ trống bị rời rạc. Ví dụ có lỗ 3 và 4 block nhưng cần 5 block liền nhau. Cấp phát liên tục có thể thất bại dù tổng còn 7 block. Liên kết và chỉ mục cho phép dùng block rời nên tránh yêu cầu liền nhau đối với dữ liệu.

Extent mô tả một đoạn block liên tiếp. Một tập tin có thể dùng nhiều extent để dễ mở rộng. Hệ thống theo dõi block trống bằng bitmap, danh sách hoặc cách khác. Phân biệt **dung lượng dành cho metadata** với **phần dư trong block cấp phát**, dù cả hai đều ảnh hưởng dung lượng hữu ích.

3. Đọc số hex và little-endian

Một byte có 8 bit và viết bằng hai chữ số hex. Ví dụ 0x0F = 15, 0x10 = 16, 0x20 = 32. Với trường số nhiều byte **little-endian**, byte thấp nằm trước. Ví dụ bytes `00 02` biểu diễn 0x0200 = 512. Bytes `39 30 00 00` biểu diễn 0x00003039 = 12.345. Chuỗi ký tự ASCII đọc theo thứ tự hiển thị, chẳng hạn `46 41 54` là FAT. UTF-16LE lưu từng đơn vị mã little-endian, chẳng hạn `48 00` là H.

Trong bài, dùng **offset** để chỉ vị trí byte trong một cấu trúc và **sector number** để chỉ thứ tự sector trong volume. Một **cluster** gồm một hoặc nhiều sector liên tiếp. Công thức chuyển phải dùng đúng đơn vị. Ví dụ sector 127 với 512 B/sector bắt đầu ở offset 127 × 512 = 65.024 B = 0xFE00.

4. Bố cục FAT và BPB

Mô hình FAT12/16 thường dùng trong đề và bài tập của lớp:

```
sector 0
[ Reserved (Boot Sector) ][ FAT1 ][ FAT2 ][ RDET ][ Data ]
                                          |       |
                                  Mục ở thư mục gốc  Cluster 2, 3, ...
```

Volume là miền lưu trữ được định dạng. **BPB (BIOS Parameter Block)** trong boot sector chứa tham số bố cục. **FAT (File Allocation Table)** ánh xạ cluster sang cluster kế hoặc mã trạng thái. **RDET (Root Directory Entry Table)** là vùng mục thư mục gốc cố định trong FAT12/16. Thư mục con lưu các mục trong vùng Data. Với FAT32, root directory cũng nằm trong chuỗi cluster vùng Data, bắt đầu tại BPB_RootClus, nên công thức cho vùng RDET cố định có giá trị 0.

Offset hex	Số byte	Trường
0B	2	Bytes mỗi sector (BPS)
0D	1	Sectors mỗi cluster (SPC)
0E	2	Số sector reserved (R)
10	1	Số bản FAT (F)
11	2	Số mục thư mục gốc (E), 0 với FAT32
13	2	Tổng sector dạng 16 bit
16	2	Số sector của mỗi FAT12/16 (S)
1C	4	Số sector trước phân vùng, dùng khi đổi sang vị trí tuyệt đối trên thiết bị
20	4	Tổng sector dạng 32 bit nếu trường 16 bit bằng 0
24	4	Số sector mỗi FAT khi dùng BPB FAT32 và trường 16 bit bằng 0
2C	4	Cluster gốc trong BPB FAT32

Tính theo thứ tự. Mỗi entry thư mục có 32 byte. Làm tròn lên khi đổi kích thước vùng RDET sang sector, vì entry cuối vẫn cần chỗ chứa. Làm tròn xuống khi đếm cluster đầy đủ.

```
RootDirSectors = ceil(32 × E / BPS)
FirstFAT1      = R
FirstFAT2      = R + S             (nếu có hai FAT)
FirstRoot      = R + F × S         (vùng RDET cố định FAT12/16)
FirstData      = R + F × S + RootDirSectors
CountClusters  = floor((TotalSectors - FirstData) / SPC)
Sector(c)      = FirstData + (c - 2) × SPC
ByteOffset     = Sector × BPS
```

Ví dụ từ handout. BPS = 512, SPC = 1, R = 1, F = 2, S = 9, E = 224, TotalSectors = 2.880. RDET có 224 × 32 / 512 = 14 sector. FAT1 bắt đầu tại 1, FAT2 tại 10, RDET tại 19, Data tại 33. Cluster 5 bắt đầu ở sector 36. Số cluster dữ liệu là 2.847, thuộc FAT12.

Xác định loại FAT. Theo đặc tả, CountClusters < 4.085 là FAT12, từ 4.085 đến 65.524 là FAT16, từ 65.525 là FAT32. Chuỗi nhãn ở BPB có thể hỗ trợ nhận diện trong bài nhưng phép phân loại dùng số cluster. Cluster 0 và 1 dành riêng. Đối với FAT16, mục tại byte offset 2c tính từ đầu bảng FAT là mục của cluster c. Với FAT32, đọc 4 byte và xét 28 bit thấp. Với FAT12, hai entry đóng gói trong 3 byte. Đọc word little-endian tại offset floor(3c/2), lấy 12 bit thấp nếu c chẵn, dịch phải 4 bit nếu c lẻ.

5. Từ mục thư mục đến nội dung file

Trong FAT16, mục FAT bằng 0 chỉ cluster trống, 0xFFF7 chỉ cluster lỗi và 0xFFF8-0xFFFF chỉ cuối chuỗi (EOC). Với FAT12, các mã tương ứng là 0x000, 0xFF7 và 0xFF8-0xFFF. Với FAT32, xét 28 bit thấp, mã lỗi là 0x0FFFFFF7 và EOC từ 0x0FFFFFF8. Một mục hợp lệ có thể chứa số cluster tiếp theo. Khi lần chuỗi trong dữ liệu bất thường, kiểm tra giới hạn và cluster đã thăm để nhận ra vòng lặp.

Ví dụ entry chứa cluster đầu 5 và kích thước 1.300 B, mỗi cluster 512 B. FAT[5] = 8, FAT[8] = 9, FAT[9] = 0xFFFF. Tập tin có chuỗi **5** → **8** → **9** → **EOC**. Nó cần ceil(1.300 / 512) = 3 cluster, và 236 B ở cuối cluster cuối nằm ngoài dữ liệu tập tin. Nếu chỉ muốn biết tập tin có bao nhiêu byte hữu ích, dùng kích thước trong entry.

6. Mục tên ngắn và tên dài

Offset hex	Số byte	Trường của mục tên ngắn (SFN)
00	11	Tên 8 ký tự + phần mở rộng 3 ký tự. Điền space 0x20 nếu thiếu.
0B	1	Thuộc tính. Read-only 01, hidden 02, system 04, volume label 08, directory 10, archive 20.
0C	1	Trường dành riêng/NT, đặt 0 trong bài mẫu
0D	1	Phần lẻ thời điểm tạo, độ phân giải 10 ms trong cách mã hóa VFAT thông dụng
0E	2	Giờ tạo
10	2	Ngày tạo
12	2	Ngày truy cập gần nhất
14	2	Phần cao của cluster đầu, 0 với FAT12/16
16	2	Giờ sửa
18	2	Ngày sửa
1A	2	Phần thấp của cluster đầu
1C	4	Kích thước file theo byte, 0 cho mục thư mục

Byte đầu `E5` đánh dấu mục đã giải phóng. Byte đầu `00` báo kết thúc danh sách mục theo quy tắc đọc FAT chuẩn. Với ảnh hex bài tập mô phỏng có các hàng trống trước mục được in rõ, phải nói rõ giả định khi đọc như dữ liệu bài tập. Tên ngắn 8.3 lưu dấu chấm ngầm, chẳng hạn `FILE TXT` là FILE.TXT. Trong thư mục con, `.` chỉ thư mục hiện tại và `..` chỉ thư mục cha.

```
date = ((year - 1980) << 9) | (month << 5) | day
time = (hour << 11) | (minute << 5) | floor(second / 2)
```

Giờ sửa lưu theo bước 2 giây. Giờ tạo có thêm byte 0D, nên một giây lẻ có thể được biểu diễn bằng phần bổ sung 100 đơn vị 10 ms. Mã hóa ngày 02/02/2020 cho kết quả (40 × 512) + (2 × 32) + 2 =

0x5042, lưu `42 50`. Giờ 01:23:45 cho 0x0AF6, lưu `F6 0A`. Ở giờ sửa, trường này biểu diễn 01:23:44. Ở giờ tạo, byte 0D = 0x64 bổ sung 1 giây nếu giả sử không có phần nhỏ hơn giây.

LFN (long file name). Mỗi entry dài chứa 13 đơn vị UTF-16LE chia thành nhóm 5 + 6 + 2 ở offset 01, 0E và 1C. Byte 0B là 0F, byte 0C là 00, byte 0D chứa checksum tên ngắn, trường cluster tại 1A là 00 00. Các entry LFN đứng ngay trước SFN và được lưu theo thứ tự N, …, 1. Entry N có bit 0x40. Nếu mảnh cuối chưa đầy, dùng U+0000 kết thúc và U+FFFF lấp chỗ còn lại. Tên dài vừa khít bội số 13 không cần thêm ô kết thúc.

Checksum tên ngắn. Khởi tạo s = 0. Với mỗi byte b trong 11 byte SFN, xoay s sang phải một bit rồi cộng b và giữ 8 bit thấp. Mọi entry LFN của cùng một tên dùng checksum đó. Tên ngắn sinh ra còn phụ thuộc va chạm tên trong thư mục. Hãy ghi quy ước alias nếu đề không cho.

7. Xác định vùng thay đổi khi thao tác

Các đáp án dưới đây giả sử FAT12/16 có hai bảng đồng bộ, thao tác trong cùng volume và thư mục đích còn chỗ, trừ khi nêu khác. Xét riêng cập nhật chuỗi cluster, vị trí entry, và dữ liệu file. “Data thay đổi” có thể là mục của thư mục con nằm trong Data.

Thao tác	FAT1, FAT2	RDET	Data
Xóa file có cluster ở root	Giải phóng chuỗi	Đánh dấu entry	Nội dung file thường còn trên đĩa đến khi bị ghi đè
Xóa file có cluster ở thư mục con	Giải phóng chuỗi	Thông thường không đổi	Đánh dấu entry của thư mục con
Move file giữa hai thư mục con	Giữ chuỗi file	Thông thường không đổi	Xóa entry cũ và tạo entry mới
Move file từ thư mục con ra root	Giữ chuỗi file	Tạo entry mới	Đánh dấu entry cũ trong thư mục con
Tạo file không rỗng ngay tại root	Cấp cluster	Tạo entry	Ghi nội dung file
Chép file không rỗng thành bản mới tại root	Cấp chuỗi riêng	Tạo entry mới	Ghi nội dung bản sao

Nếu thư mục con đích phải mở rộng, phải cấp thêm cluster cho **thư mục**, vì vậy FAT có thể đổi dù chuỗi file giữ nguyên. Di chuyển giữa hai volume thường phải sao chép rồi xóa bản cũ. Trường ngày giờ hoặc thuộc tính có thể được cập nhật tùy driver, nên bài cần nêu mô hình đang xét. Xóa thông thường khác thao tác ghi đè để xóa an toàn.

Tự luyện

Bài 1. BPS = 512, R = 1, hai FAT mỗi bảng 4 sector, E = 64, SPC = 2. Tìm sector đầu RDET, Data và cluster 7.

Giải. RDET dài ceil(64 × 32 / 512) = 4 sector. FirstRoot = 1 + 8 = 9. FirstData = 13. Cluster 7 ở 13 + (7 − 2) × 2 = **23**.

Bài 2. File 0 byte ở thư mục con không có cluster. Xóa file, giả sử không cập nhật thêm metadata của cha. Vùng nào phải đổi?

Giải. Data đổi tại entry của thư mục con. Không có chuỗi cluster để giải phóng. RDET không cần đổi.

Nguồn đối chiếu · 4a File system, slide 4-41. 4b Implement File System, slide 4-33. Hang out, trang 1-2. FAT hw 1-2. Operating System Concepts 10e, chương 13-15. Microsoft FAT Specification, mục 3-7.

05 · Tiến trình, luồng và fork

Sau chương này, bạn có thể đọc sơ đồ trạng thái, phân biệt tiến trình với luồng và đếm số lần lệnh được thực thi qua fork().

1. Chương trình và tiến trình

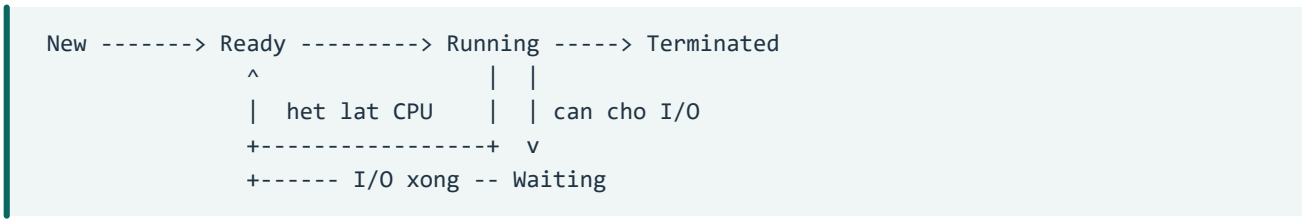
Chương trình (program) là mã và dữ liệu được lưu trữ. **Tiến trình (process)** là một thực thể đang thực thi chương trình, gồm không gian địa chỉ, trạng thái CPU và các tài nguyên như tập tin đang mở. Mở hai cửa sổ có thể tạo hai tiến trình hoặc dùng chung một tiến trình tùy thiết kế ứng dụng.

Vùng điển hình	Chứa gì?
Text	Lệnh máy của chương trình
Data / BSS	Biến toàn cục hoặc tĩnh. BSS dùng cho dữ liệu tĩnh khởi tạo bằng 0 hoặc chưa khởi tạo tường minh.
Heap	Dữ liệu được cấp phát động, chẳng hạn bằng malloc
Stack	Khung lời gọi hàm, địa chỉ trả về và nhiều biến cục bộ

Đây là mô hình tổ chức điển hình. Trình biên dịch có thể giữ một biến trong thanh ghi. Khung lời gọi (activation record) giúp tìm tham số, biến cục bộ và trạng thái để trở về hàm gọi. Stack pointer theo dõi vị trí hiện thời của stack theo quy ước kiến trúc.

2. Trạng thái, PCB và chuyển ngữ cảnh

Trạng thái	Diễn giải
New	OS đang tạo tiến trình
Ready	Có thể chạy, đang đợi CPU
Running	Đang chạy trên CPU
Waiting / blocked	Đang đợi sự kiện, ví dụ I/O
Terminated	Đã kết thúc thực thi



Ready → Running khi được cấp CPU. Running → Ready khi bị thu hồi CPU. Running → Waiting khi cần đợi sự kiện. Waiting → Ready khi sự kiện hoàn tất. Một tiến trình vừa hoàn tất I/O có thể được chọn ngay sau khi về Ready, nhưng hoàn tất I/O và được chọn chạy là hai bước khác nhau.

PCB (Process Control Block) là cấu trúc kernel dùng để quản lý tiến trình. Nó lưu hoặc tham chiếu định danh, trạng thái, thông tin lập lịch, tài nguyên I/O và không gian địa chỉ. **Context switch** lưu trạng thái đang chạy rồi nạp trạng thái của thực thể khác. Các công việc như lưu thanh ghi và làm nóng lại cache có chi phí. **CPU burst** là khoảng dùng CPU giữa các lần chờ hoặc kết thúc. Việc I/O-bound thường có nhiều burst ngắn. Việc CPU-bound thường dành nhiều thời gian cho tính toán.

3. Tạo, thay chương trình và chờ

Lời gọi	Hành vi trong mô hình Unix/POSIX
fork()	Tạo tiến trình con. Cha nhận PID con, con nhận 0. Cha nhận giá trị âm nếu tạo thất bại.
exec…()	Thay ảnh chương trình trong tiến trình hiện có nếu thành công.
wait() / waitpid()	Đợi hoặc thu trạng thái kết thúc của tiến trình con theo lựa chọn của lời gọi.
exit()	Kết thúc tiến trình theo đường thư viện C, có xử lý như flush stream.
_exit()	Kết thúc tiến trình mà không flush các buffer stdio của thư viện C.

Sau fork thành công, cả cha và con tiếp tục từ điểm ngay sau lời gọi. Giá trị bộ nhớ ban đầu của con được sao chép theo ngữ nghĩa. Hai không gian địa chỉ độc lập đối với các vùng riêng. OS có thể dùng **copy-on-write** để trì hoãn sao chép page đến lúc ghi. Một số tài nguyên kernel có thể vẫn được chia sẻ qua descriptor thừa kế. Ví dụ tiến trình cha và con có thể tham chiếu cùng một open file description và chia sẻ vị trí file.

Ví dụ 1. Nếu mọi fork thành công, hai lần fork nối tiếp tạo 1 → 2 → 4 tiến trình. Có 3 tiến trình con được tạo tổng cộng. Lệnh đứng sau cả hai fork chạy 4 lần. Lệnh đứng trước cả hai fork chạy 1 lần.

```
int main(void) {
    fork();                 // 1 tien trinh goi
    fork();                 // 2 tien trinh goi
    puts("bye");            // 4 tien trinh goi
    return 0;
}
```

Ví dụ 2. Sau `if (fork() == 0) fork();` có 3 tiến trình nếu cả hai lời gọi thành công. Chỉ con từ lần đầu đi vào thân if. Không thể áp dụng 2^n cho mọi đoạn có n chữ fork. Hãy vẽ cây và ghi điều kiện trên từng nhánh.

Bộ đệm stdout. Đếm số lần gọi printf và đếm số lần chữ xuất hiện có thể cho kết quả khác. Nếu `printf("hello")` đứng trước fork và chữ vẫn nằm trong bộ đệm thư viện, con thừa hưởng một bản sao bộ đệm. Khi từng tiến trình kết thúc theo đường flush, mỗi bản sao có thể xuất hiện. Trên hệ thống thường gặp, với hai fork thành công và stdout còn đệm, mỗi tiến trình có thể flush một chuỗi `hellobye`. Nếu stdout đã được flush trước fork, phần hello đã in chỉ xuất hiện một lần và bye xuất hiện bốn lần. Trong mã thực tế cần phối hợp truy cập stream khi fork, chẳng hạn flush trước fork. Thứ tự tiến trình chạy tùy scheduler.

4. Luồng và trao đổi giữa tiến trình

Luồng (thread) là một dòng thực thi. Các luồng cùng tiến trình thường chia sẻ code, heap và tài nguyên như tập tin đang mở. Mỗi luồng có PC, thanh ghi và stack riêng. Nhiều luồng giúp ứng dụng vẫn phản hồi khi một luồng chờ I/O và có thể khai thác nhiều lõi. **Đồng thời (concurrency)** là nhiều việc cùng tiến triển trong một khoảng thời gian. **Song song (parallelism)** là nhiều việc thực thi tại cùng một thời điểm. Một lõi có thể tạo đồng thời bằng cách thay phiên, còn nhiều lõi có thể thực thi song song.

Ở mô hình many-to-one, nhiều luồng user dùng chung một luồng kernel. One-to-one ánh xạ từng luồng user sang từng luồng kernel. Many-to-many ghép nhiều luồng user lên một tập luồng kernel. Việc quản lý luồng có thể được thực hiện qua thư viện, hệ điều hành hoặc cả hai.

IPC	Cách hoạt động	Điểm cần chú ý
Shared memory	Các tiến trình ánh xạ một vùng nhớ chung	Truy cập dữ liệu trực tiếp sau thiết lập. Cần đồng bộ khi cập nhật chung.
Message passing	Gửi/nhận thông điệp qua cơ chế OS	Có thể gửi trực tiếp hoặc qua mailbox. Lệnh gửi/nhận có thể blocking hoặc non-blocking.

Signal là thông báo từ OS hoặc tiến trình khác gửi đến tiến trình hoặc luồng theo quy tắc hệ thống. Nó có thể gọi một handler, bị bỏ qua khi được phép hoặc thực hiện hành vi mặc định. Ví dụ kernel có thể gửi tín hiệu khi chương trình truy cập bộ nhớ lỗi. Interrupt phần cứng được CPU và kernel xử lý ở lớp khác.

Ví dụ producer ghi dữ liệu vào bộ đệm vòng, consumer đọc ra. Với chỉ số in và out, một mô hình quy ước rỗng khi in = out, đầy khi (in + 1) mod N = out, nên chứa tối đa N − 1 phần tử. Điều kiện đầy/rỗng và quyền cập nhật chỉ số phải phù hợp với số producer/consumer và mô hình đồng bộ.

Tự luyện

Bài 1. Một chương trình gọi fork() ba lần liên tiếp, mọi lần thành công. Có bao nhiêu tiến trình, bao nhiêu tiến trình mới và bao nhiêu lần gọi fork thực sự?

Giải. Có 8 tiến trình tổng cộng, 7 tiến trình mới. Số lần gọi thật là 1 + 2 + 4 = 7.

Bài 2. Ứng dụng đang đợi đọc đĩa. Khi đĩa báo hoàn tất, tiến trình ở trạng thái nào trước khi được cấp CPU?

Giải. Waiting → Ready. Scheduler chọn thì tiến trình mới chuyển Ready → Running.

Nguồn đối chiếu · 5 processes, slide 2-45. Operating System Concepts 10e, chương 3-4 và mục 10.3.

06 · Điều phối CPU

Sau chương này, bạn có thể tự vẽ Gantt, tính thời gian chờ, phản hồi và hoàn thành, rồi giải thích vì sao kết quả phụ thuộc thuật toán và tải.

1. Scheduler, dispatcher và tiêu chí

CPU scheduler chọn thực thể sẵn sàng trong ready queue. **Dispatcher** giao quyền CPU cho thực thể đã chọn bằng các thao tác như chuyển ngữ cảnh, đổi chế độ và tiếp tục tại lệnh phù hợp. Lập lịch **không thu hồi (non-preemptive)** thường để burst chạy tới khi kết thúc hoặc tự nhường. Lập lịch **có thu hồi (preemptive)** có thể dừng thực thể đang chạy vì hết lát hoặc vì xuất hiện ứng viên ưu tiên hơn.

Đại lượng	Ký hiệu và cách tính
Arrival time	A, thời điểm tiến trình đến
CPU burst	B, thời gian CPU cần để xử lý burst
First start	F, lần đầu được cấp CPU
Completion time	C, thời điểm hoàn tất trên trục thời gian
Turnaround time	T = C − A, thời gian từ đến đến hoàn tất
Response time	R = F − A trong bài CPU mô hình hóa lần đầu chạy là lần đáp ứng đầu tiên
Waiting time	W, tổng thời gian ở ready queue. Với một burst và không I/O, W = T − B.

Với nhiều lần chờ I/O, đừng tính mọi thời gian ngoài CPU vào waiting. Throughput đếm số việc hoàn tất trên đơn vị thời gian. CPU utilization đo phần thời gian CPU bận. Khi dùng trung bình, tính giá trị cho từng tiến trình rồi mới chia cho số tiến trình.

2. Thuật toán và cách làm

Thuật toán	Quy tắc	Chi phí hoặc hạn chế
FCFS	Chọn theo thứ tự vào ready queue. Không thu hồi.	Việc ngắn có thể đợi việc dài (convoy effect).
SJF	Chọn CPU burst ngắn nhất trong nhóm ready. Phiên bản này không thu hồi.	Phải biết hoặc ước lượng burst. Tối ưu trung bình chờ trong mô hình các job đến đồng thời, một burst và không tính overhead.
SRTF	Chọn thời gian chạy còn lại ngắn nhất. Có thu hồi.	Xem xét lại khi tiến trình mới đến.
Priority	Chọn ưu tiên cao nhất theo quy ước đề. Có thể thu hồi hoặc không.	Ưu tiên thấp có thể bị đói CPU. Aging tăng ưu tiên khi đợi lâu.
RR	Chạy tối đa quantum q. Nếu chưa xong và còn ready, đưa về cuối hàng đợi.	q nhỏ giúp chia lượt sớm nhưng tăng chuyển ngữ cảnh. q đủ lớn làm RR giống FCFS trong mô hình một burst.

Cách làm. Đặt t tại thời điểm có sự kiện. Đưa vào ready queue những tiến trình đã đến. Chọn theo thuật toán, chạy đến sự kiện tiếp theo có thể đổi quyết định. Viết các khoảng idle nếu không có tiến trình ready. Nếu hết quantum đúng lúc có tiến trình mới đến, ghi quy ước xử lý và dùng nhất quán.

Ước lượng burst. Nếu burst vừa đo là t(n) và dự đoán cũ là τ(n), dự đoán mới có thể tính τ(n + 1) = α × t(n) + (1 − α) × τ(n). Với α = 0,5, dự đoán cũ 8 và vừa chạy 4, dự đoán tiếp theo là 6.

3. Ví dụ giải đủ bốn thuật toán

Giả sử chỉ một CPU, mỗi tiến trình có một burst, không tính overhead. Nếu hòa, chọn theo thứ tự vào hàng đợi rồi theo số hiệu. Với RR, tại cùng một mốc, nhận tiến trình mới trước khi đưa tiến trình hết quantum về cuối hàng.

Tiến trình	A	B
P1	0	7
P2	2	4
P3	4	1
P4	5	4

```
FCFS  0 | P1 | 7 | P2 | 11 | P3 | 12 | P4 | 16
SJF   0 | P1 | 7 | P3 |  8 | P2 | 12 | P4 | 16
SRTF  0 | P1 | 2 | P2 |  4 | P3 |  5 | P2 | 7 | P4 | 11 | P1 | 16
RR    0-2 P1, 2-4 P2, 4-6 P1, 6-7 P3, 7-9 P2,
q=2   9-11 P4, 11-13 P1, 13-15 P4, 15-16 P1
```

Ở SJF, P1 là ứng viên duy nhất lúc 0, nên chạy đến 7. Lúc đó P3 ngắn nhất. Ở SRTF, khi P2 đến lúc 2, P1 còn 5 nên P2 giành CPU. P3 đến lúc 4 với 1, ngắn hơn 2 còn lại của P2. Ở RR, P3 đến đúng t = 4. P1 đã chờ từ t = 2 nên tiếp tục đứng trước P3.

Thuật toán	C theo P1-P4	W theo P1-P4	W trung bình	T trung bình	R trung bình
FCFS	7, 11, 12, 16	0, 5, 7, 7	4,75	8,75	4,75
SJF	7, 12, 8, 16	0, 6, 3, 7	4	8	4
SRTF	16, 7, 5, 11	9, 1, 0, 2	3	7	0,5
RR q = 2	16, 9, 7, 15	9, 3, 2, 6	5	9	1,5

Ví dụ P2 trong SRTF. A = 2, B = 4, F = 2, C = 7. T = 7 − 2 = 5, R = 2 − 2 = 0, W = 5 − 4 = 1. Đọc lại Gantt thấy P2 chạy hai đoạn 2-4 và 5-7, chỉ chờ từ 4 đến 5.

Cách trả lời "tốt nhất". RR thường phù hợp cho phản hồi tương tác vì chia lượt sớm. SJF cho turnaround trung bình tối ưu trong mô hình các job đến cùng lúc và burst biết trước, vì tổng burst cố định và chờ trung bình nhỏ nhất. Các thuật toán có thể cho thứ hạng khác khi thay tải. Bảng trên là một phản ví dụ nếu khẳng định RR luôn có response trung bình nhỏ nhất.

4. Điều phối nhiều hàng đợi và các mở rộng

Multilevel queue chia việc vào các hàng đợi, có thể dành RR cho tương tác và FCFS cho batch. **Multilevel feedback queue (MLFQ)** cho phép đổi mức. Ví dụ Q0 dùng q = 8, Q1 dùng q = 16 và Q2 dùng FCFS. Việc mới vào Q0. Việc dùng hết lát mà chưa xong có thể bị hạ xuống Q1, rồi Q2. Có thể tăng mức cho việc chờ lâu để giảm starvation. Một lịch quan sát được còn phụ thuộc quy tắc điều phối giữa các hàng đợi.

Điều phối thời gian thực. Job có deadline. RMS ưu tiên chu kỳ ngắn hơn trong mô hình tác vụ định kỳ. EDF chọn deadline tuyệt đối gần nhất. Với n tác vụ định kỳ độc lập, có thể thu hồi, deadline bằng chu kỳ và bỏ qua overhead trên một CPU, RMS có điều kiện đủ $\Sigma(C_i/T_i) \le n(2^{(1/n)} - 1)$. Vượt cận đủ cần phân tích tiếp. Trong mô hình này, EDF có điều kiện khả thi $\Sigma(C_i/T_i) \le 1$. Chẳng hạn (C,T) là (1,4) và (2,5), tổng 0,65, nhỏ hơn cả cận RMS cho hai tác vụ lẫn 1.

Nhiều CPU. Cân bằng tải chuyển việc để tránh lõi quá bận. Processor affinity có thể giữ việc trên một lõi để tận dụng cache. Hai mục tiêu có thể cạnh tranh. Thông tin ưu tiên, tải và chính sách UNIX truyền thống trong slide là các ví dụ triển khai theo từng hệ thống.

Tự luyện

Bài 1. Ba tiến trình cùng đến lúc 0, B lần lượt là 5, 1, 2. Tính W trung bình của FCFS theo thứ tự 1, 2, 3 và SJF.

Giải. FCFS chạy P1, P2, P3 với W = 0, 5, 6, trung bình 11/3. SJF chạy P2, P3, P1 với W theo thứ tự chạy = 0, 1, 3, trung bình 4/3.

Bài 2. P đến lúc 3, chạy lần đầu lúc 7, kết thúc lúc 15 và dùng CPU tổng 6. Một burst, không chờ I/O. Tính T, R, W.

Giải. T = 12, R = 4, W = 6. Nếu P bị thu hồi ở giữa, R vẫn dùng mốc đầu tiên. W bao gồm cả khoảng chờ sau khi bị thu hồi.

Nguồn đối chiếu · 6 Dieu phoi CPU, slide 8-48. 6 Process scheduling, slide 3-42. Operating System Concepts 10e, chương 5.

07 · Đồng bộ và bộ nhớ

Race condition đã xuất hiện trong đề 2024-2025. Phần sau về deadlock và bộ nhớ giúp nối các ý trong slide và ôn thêm khi lớp yêu cầu.

1. Race condition và miền găng

Race condition xảy ra khi nhiều dòng thực thi truy cập dữ liệu chung và kết quả phụ thuộc thứ tự xen kẽ. **Miền găng (critical section)** là đoạn cần bảo vệ vì truy cập trạng thái dùng chung. Lệnh nhìn như một bước ở mã nguồn có thể gồm nhiều bước máy. Ví dụ `visitor++` gồm đọc, cộng và ghi trong mô hình bài thi.

Bước	Luồng A	Luồng B	Giá trị chung
1	Đọc 0		0
2		Đọc 0	0
3	Cộng rồi ghi 1		1
4		Cộng rồi ghi 1	1

Cả hai lượt đã hoàn thành nhưng một lượt tăng bị mất. Dùng mutex bao quanh cả phép đọc-sửa-ghi, hoặc dùng phép tăng atomic, có thể bảo đảm mỗi lượt tăng đóng góp đúng 1. Trong C/C++ thực tế, các truy cập không atomic đồng thời có ghi mà thiếu đồng bộ tạo data race và có thể gây hành vi không xác định. Các kết quả 1, 2, 3 ở đề cũ dùng mô hình xen kẽ đơn giản hóa.

Ba yêu cầu điển hình của giải pháp miền găng gồm **mutual exclusion** (mỗi lúc tối đa một bên vào), **progress** (nếu miền găng trống và có bên muốn vào, quyết định được thực hiện) và **bounded waiting** (có giới hạn số lần bị vượt lượt theo giả thiết). Starvation là một bên chờ quá lâu trong khi hệ thống vẫn có thể tiếp tục.

2. Khóa và semaphore

Một biến cờ kiểu “kiểm tra rồi gán” không đủ nếu hai bước có thể xen nhau. **Test-and-set** thực hiện đọc giá trị cũ và đặt cờ như một thao tác nguyên tử. Spinlock chờ bằng vòng lặp và dùng CPU. Khóa blocking cho phép bên chờ ngủ. Vô hiệu ngắt cục bộ có thể ngăn ngắt trên lõi đó nhưng tự nó không ngăn lõi khác truy cập dữ liệu chung.

Peterson dùng hai cờ muốn vào và một biến lượt cho hai bên. Mỗi bên đặt cờ của mình, nhường lượt cho bên kia và đợi nếu bên kia vừa muốn vào vừa được ưu tiên. Lập luận cổ điển giả định bộ nhớ tuần tự và đọc/ghi thích hợp. Khi triển khai trên phần cứng/ngôn ngữ hiện đại cần cơ chế atomic và thứ tự bộ nhớ tương ứng.

Semaphore là một bộ đếm đồng bộ với hai thao tác nguyên tử `wait/down` và `signal/up`. Down lấy một đơn vị nếu có, hoặc đợi. Up trả một đơn vị hoặc đánh thức bên chờ theo quy tắc hiện thực. Semaphore đếm tài nguyên. Mutex bảo vệ quyền vào miền găng và thường có khái niệm sở hữu.

Ví dụ bộ đệm N ô. Dùng empty = N, full = 0, mutex = 1. Producer chờ empty trước rồi lấy mutex, đưa phần tử vào, nhả mutex và báo full. Consumer chờ full trước rồi lấy mutex, lấy phần tử ra, nhả mutex và báo empty. Nếu producer giữ mutex rồi chờ empty khi bộ đệm đầy, consumer có thể không lấy được mutex để giải phóng ô.

3. Deadlock

Deadlock xảy ra khi một nhóm chờ lẫn nhau và không bên nào có thể tiến lên. Bốn điều kiện kinh điển là loại trừ tương hỗ, giữ và chờ, không thu hồi cưỡng bức tài nguyên, và chờ vòng. Trong dining philosophers, nếu mọi người cầm một nĩa và chờ nĩa còn lại theo một vòng, không ai ăn được. Có thể quy định thứ tự lấy tài nguyên để phá chờ vòng, hoặc dùng cơ chế chỉ cấp khi có thể lấy cả hai. Starvation có thể tồn tại dù một số người vẫn ăn được.

Phòng ngừa deadlock phá một điều kiện cần. Tránh deadlock chỉ cấp khi vẫn có lịch hoàn tất an toàn theo thông tin về nhu cầu. Phát hiện và phục hồi cho phép xảy ra rồi tìm cách phá thế kẹt. Với nhiều bản sao của một loại tài nguyên, một chu trình trên đồ thị cấp phát tài nguyên là dấu hiệu cần phân tích thêm.

4. Địa chỉ logic và phân trang

CPU sinh **địa chỉ logic/ảo**. **MMU** (Memory Management Unit) dịch địa chỉ và áp dụng quyền truy cập. Trong mô hình base/limit, kiểm tra 0 ≤ offset < limit trước khi cộng base. Nếu base = 4.000 và limit = 1.000, offset 420 ánh xạ thành 4.420. Offset 1.000 vượt giới hạn.

Trong cấp phát RAM liên tục, first-fit lấy lỗ đầu đủ lớn, best-fit lấy lỗ nhỏ nhất đủ lớn và worst-fit lấy lỗ lớn nhất. Phân mảnh ngoại vi phát sinh khi tổng vùng rảnh đủ nhưng các lỗ không liền nhau. **Swapping** chuyển dữ liệu của tiến trình giữa RAM và bộ nhớ thứ cấp. **Dynamic loading** nạp phần mã khi được dùng, còn **dynamic linking** trì hoãn liên kết với thư viện đến thời điểm thích hợp khi chạy hoặc nạp.

Paging chia không gian ảo thành page và RAM thành frame cùng kích thước. Bảng trang ánh xạ page sang frame, còn offset trong page giữ nguyên. Page table entry thường chứa quyền và thông tin trạng thái. **TLB** cache kết quả dịch địa chỉ. TLB miss có thể chỉ cần tra bảng trang. Page fault là sự kiện khác, chẳng hạn page hợp lệ chưa hiện diện. Với bảng trang nhiều cấp, chỉ cần cấp các bảng con của vùng địa chỉ được sử dụng. Bảng băm và bảng đảo là các phương án tổ chức khác.

Ví dụ địa chỉ ảo 2.500 với page 1.024 byte. Page = floor(2.500/1.024) = 2, offset = 452. Nếu page 2 ánh xạ frame 7, địa chỉ vật lý = 7 × 1.024 + 452 = **7.620**. Với page size 2^k, k bit thấp là offset. Paging tránh yêu cầu frame liền kề nhưng có thể dư chỗ bên trong page cuối.

Segmentation chia theo đơn vị logic có thể khác kích thước như code, stack, data. Địa chỉ gồm số segment và offset. Tra base/limit tương ứng, kiểm tra offset, rồi cộng base. Phân đoạn cấp phát liên tục theo từng segment có thể gặp phân mảnh ngoại vi.

5. Bộ nhớ ảo và thay trang

Demand paging nạp page khi cần. Một page fault do page hợp lệ chưa hiện diện có thể được xử lý bằng cách tìm frame, thay page nếu cần, nạp page, cập nhật ánh xạ và thử lại lệnh. Nếu page bị đẩy ra đã sửa (dirty), hệ thống có thể phải ghi lại trước. Vi phạm quyền hoặc địa chỉ không hợp lệ được xử lý khác. Với tỷ lệ fault p, mô hình đơn giản dùng EAT ≈ (1 − p) × t_hit + p × t_fault, trong đó thời gian fault đã tính cả xử lý và tiếp tục.

Thuật toán	Chọn page bị đẩy
FIFO	Page vào RAM sớm nhất
OPT	Page còn lâu nhất mới dùng, hoặc không còn dùng. Dùng để so sánh vì cần biết tương lai.
LRU	Page có lần truy cập gần nhất xa nhất về quá khứ
Clock / second chance	Duyệt vòng, cho page có bit tham chiếu một cơ hội nữa rồi xóa bit

Ví dụ có 3 frame, chuỗi 1, 2, 3, 1, 4, 1. FIFO nạp 1, 2, 3 rồi trúng 1. Khi nạp 4, FIFO đẩy 1, nên lần truy cập 1 cuối lại fault. Tổng **5**. LRU giữ 1 vì vừa dùng và đẩy 2 khi nạp 4. Tổng **4**. Với FIFO, tăng frame đôi khi làm tăng fault (Belady's anomaly). LRU và OPT là các thuật toán stack và không có hiện tượng này trong mô hình chuẩn.

Thrashing là tình trạng hệ thống dành nhiều thời gian nạp/đẩy page vì tập page đang dùng không đủ chỗ. **Working set** ước lượng các page được tham chiếu trong một cửa sổ gần đây. Cấp frame có thể chia theo kích thước/ưu tiên và thay page cục bộ hoặc toàn cục. Tệp ánh xạ bộ nhớ cho phép truy cập nội dung qua địa chỉ ảo. Kích thước page lớn tăng phạm vi mỗi mục TLB, đồng thời có thể làm tăng phần dư và chi phí nạp dữ liệu không cần.

Tự luyện

Bài 1. Semaphore full đang bằng 0. Consumer down(full) trước khi lấy mutex. Producer có thể tiến lên không?

Giải. Có, consumer chờ mà chưa giữ mutex. Producer có thể lấy mutex, thêm phần tử rồi up(full).

Bài 2. RAM có 4 KiB/frame. Địa chỉ ảo 0x1234. Page tương ứng ánh xạ vào frame 5. Tìm địa chỉ vật lý.

Giải. 4 KiB = 0x1000. Page = 1 và offset = 0x234. Địa chỉ vật lý = 0x5000 + 0x234 = **0x5234**.

Nguồn đối chiếu · 7 synchronization, slide 2-24. 8 Memory management và 8 Virtual memory. Operating System Concepts 10e, chương 6-10.

08 · Lời giải ba đề giữa kỳ

Các câu hỏi được diễn giải để bạn có thể đối chiếu với ảnh đề gốc. Ở câu có dữ kiện thiếu hoặc phụ thuộc quy ước, lời giải nêu giả thiết và một cách trình bày có thể kiểm tra.

Đề 2025-2026 · 23CNTN

Ảnh đề ghi đây là bản biên soạn lại theo trí nhớ của sinh viên.

Câu 1 · Vì sao time-sharing cần interrupt?

Nếu ứng dụng đang chạy không gọi OS và không chờ I/O, kernel vẫn phải lấy lại CPU để chia lượt cho việc khác. Bộ định thời phát ngắt khi hết lát. CPU vào kernel, kernel lưu trạng thái và có thể chọn một tiến trình ready khác. Nhờ đó một chương trình tính lâu không giữ CPU vô hạn trong mô hình này.

Câu 2 · Khác biệt giữa time-sharing và multiprogramming

Đa chương giữ nhiều việc để CPU có việc làm khi việc hiện tại chờ I/O. Chia sẻ thời gian hướng đến phản hồi tương tác bằng cách luân phiên CPU thường xuyên và có thể thu hồi CPU theo bộ định thời. Ví dụ một việc dùng CPU lâu vẫn bị tạm dừng khi hết lát trong time-sharing.

Câu 3 · Khi nào polling tốt hơn interrupt cho card mạng?

Khi gói đến rất thường xuyên, polling có thể lấy nhiều gói một lần và tránh vào/ra handler cho từng gói. Khi lưu lượng thưa, ngắt thường tránh được nhiều lượt kiểm tra rỗng. Câu trả lời cần liên hệ tần suất sự kiện với chi phí xử lý ngắt và chu kỳ polling.

Câu 4 · Khi nào PIO tốt hơn DMA?

Khi truyền rất ít dữ liệu, PIO có thể dùng ít thời gian CPU hơn do ít chi phí thiết lập và xử lý hoàn tất. Với khối lớn, DMA chuyển giữa RAM và thiết bị sau khi thiết lập, giúp CPU có thể làm việc khác. Nếu đề cho chu kỳ, lập hai biểu thức rồi tìm ngưỡng như chương 2.

Câu 5 · Khi nào chuyển từ user mode sang system mode?

Khi chương trình yêu cầu system call, khi lệnh gây exception (ví dụ page fault) hoặc khi CPU đang ở user mode và nhận hardware interrupt (ví dụ timer hoặc I/O). CPU đi vào điểm xử lý kernel thích hợp. Ghi rõ sự kiện xuất phát từ đâu.

Câu 6 · Phân mảnh theo cách cấp phát file

Liên tục có thể gặp phân mảnh ngoại vi vì cần một đoạn liền. Liên kết và chỉ mục có thể dùng block rời. Khi đơn vị cấp phát có cỡ cố định, cả ba đều có thể dư phần cuối block dữ liệu, tức phân mảnh nội vi. Ví dụ file 1.100 B dùng hai block 1.024 B và dư 948 B.

Câu 7 · Những vùng nào thay đổi trong FAT?

Xét FAT12/16 như sơ đồ đề, hai bảng FAT đồng bộ, file có dữ liệu, move trong cùng volume, thư mục đích còn chỗ và xét cập nhật bắt buộc của thao tác.

Ý	Phải thay đổi	Lý do
a. Xóa file trong thư mục con của root	FAT1, FAT2, Data	Giải phóng chuỗi trong hai FAT. Entry của file nằm trong thư mục con ở Data.
b. Move file giữa hai thư mục con của root	Data	Đánh dấu entry cũ, tạo entry đích. Chuỗi cluster file giữ nguyên.
c. Hai thao tác đổi cả bốn vùng	Tạo file không rỗng ở root. Chép file thành bản mới không rỗng ở root.	Mỗi thao tác cấp cluster, tạo entry root và ghi nội dung.

Nếu phải mở rộng thư mục đích ở câu b, hai FAT cũng có thể đổi để nối cluster của thư mục. Khi xét timestamp tùy driver, cần nói rõ có tính metadata thư mục cha hay không.

Câu 8 · Khi nào đa chương phù hợp hơn time-sharing?

Các việc theo lô ít cần phản hồi ngay, thường có burst CPU ngắn và tự chặn khi chờ I/O là ví dụ. Việc khác có thể chạy lúc một việc đợi. Khi mọi việc tự nhường đủ thường xuyên, thêm nhiều lần thu hồi theo quantum có thể tăng overhead mà ít giúp mục tiêu của tải đó.

Câu 9 · Ví dụ SCAN < FCFS < SSTF với 10 cylinder

Đánh số 0-9, đầu đọc ở 5, SCAN đi hướng tăng trước. Hàng đợi theo thứ tự đến là [0, 4, 7, 9], có sẵn trước khi phục vụ. Dừng khi xử lý xong yêu cầu cuối, đo thời gian seek theo tổng dịch chuyển.

Thuật toán	Đường đi	Tổng
SCAN	5 → 7 → 9 → 4 → 0	13
FCFS	5 → 0 → 4 → 7 → 9	14
SSTF	5 → 4 → 7 → 9 → 0	15

Mỗi bước SSTF có ứng viên gần nhất duy nhất. Cả hai biên đều có yêu cầu nên ví dụ cũng không lệ thuộc vào việc có tính đoạn đi ra biên trước khi đảo chiều hay không.

Đề 2024-2025 · 22TN

Câu 1 · HĐH hiện đại cần ngắt để làm gì?

Thiết bị báo sự kiện như hoàn tất I/O để CPU xử lý đúng lúc mà vẫn có thể thực hiện công việc khác. Timer interrupt còn trao quyền lại cho kernel để thực thi điều phối có thu hồi.

Câu 2 · Khi nào interrupt tốt hơn polling?

Khi dữ liệu mới xuất hiện thưa hoặc khó đoán và CPU có công việc khác. Ngắt tránh kiểm tra trạng thái liên tục chỉ để thấy chưa có dữ liệu. Khi dữ liệu đến dày, xem lại chi phí ngắt trên mỗi sự kiện.

Câu 3 · Khi nào PIO tốt hơn DMA?

Truyền ít byte hoặc giao dịch nhỏ, khi chi phí cài đặt và hoàn tất DMA vượt chi phí CPU tự chuyển dữ liệu. Nêu rõ bạn đang so sánh thời gian CPU hay thời gian hoàn tất nếu đề cung cấp số liệu.

Câu 4 · Khi nào đa chương phù hợp hơn time-sharing?

Tải theo lô, ít yêu cầu phản hồi tương tác và có các điểm chờ I/O đủ thường xuyên. Có thể chạy việc khác lúc một việc chờ với ít lần chuyển ngữ cảnh vì hết lát. Đây là nhận xét theo mục tiêu và tải, không phải kết luận cho mọi tiến trình I/O-bound.

Câu 5 · Ứng dụng user có được truy cập vùng nhớ OS không?

Vùng chỉ dành cho kernel được bảo vệ bằng đặc quyền và quyền truy cập bộ nhớ. Lệnh user truy cập trực tiếp sẽ gây ngoại lệ. Ứng dụng yêu cầu thao tác hợp lệ thông qua system call, kernel kiểm tra tham số và thực hiện theo quyền cho phép.

Câu 6 · Liệt kê ba thuật toán CPU

Có thể chọn FCFS, SJF không thu hồi và Round Robin. FCFS theo thứ tự sẵn sàng. SJF chọn burst ngắn nhất đang ready. RR chia quantum và luân phiên theo ready queue.

Câu 7 · Thuật toán nào tốt hơn về response và turnaround?

Với lựa chọn ở câu 6, **RR** thường phù hợp cho phản hồi tương tác vì mỗi việc có cơ hội nhận lát sớm. **SJF** tối ưu turnaround trung bình khi tất cả đến đồng thời, biết burst và bỏ qua overhead. Nếu đề hỏi một bảng dữ liệu cụ thể thì vẽ Gantt và tính F − A, C − A. Kết quả response còn phụ thuộc quantum, thứ tự đến và độ dài burst. Ví dụ ba việc B = 5, 1, 2 đến lúc 0, FCFS cho R trung bình 11/3, SJF cho 4/3, RR q = 1 cho (0 + 1 + 2)/3 = 1.

Câu 8 · Chuỗi CPU 1, 2, 3, 2, 1, 3

Chuỗi cấp CPU thiếu thông tin về burst, I/O và quy tắc thu hồi, nên có thể có nhiều lịch hợp lệ. Trong mô hình ba việc cùng đến, mỗi việc chỉ có một burst, FCFS và SJF không thu hồi chỉ cấp một đoạn cho mỗi việc. RR thuần với một burst và cả ba việc đều xuất hiện hai lượt cũng sẽ giữ thứ tự vòng 1, 2, 3 trong lượt thứ hai. Một cách tái tạo chuỗi đã cho là **RR có chờ I/O**.

Cho q = 2. P1 chạy 0-1 rồi chờ I/O đến t = 4. P2 chạy 1-3 rồi xếp sau P3. P3 chạy 3-5. Tại t = 4, P1 được đưa vào hàng sau P2. P3 hết lát và về sau P1. Sau đó P2 chạy 5-6, P1 6-7 và P3 7-8, mỗi việc hoàn tất ở cuối đoạn tương ứng. Lịch quan sát được đúng 1, 2, 3, 2, 1, 3. Nếu bài yêu cầu suy ra duy nhất một thuật toán, cần thêm dữ kiện.

Câu 9 · printf("hello"), fork(), fork(), printf("bye")

Giả sử fork thành công. Lời gọi in hello thực thi một lần và lời gọi in bye thực thi bốn lần. Với stdout thường gặp còn đệm sau hello và mọi tiến trình thoát bình thường, mỗi tiến trình mang bản sao bộ đệm đó, nên có thể thấy `hellobye` bốn lần. Nếu hello đã flush trước fork, thấy hello một lần và bye bốn lần. Việc in ra môi trường nào, flush thế nào và thứ tự chạy ảnh hưởng chuỗi quan sát. Mã C chuẩn nên dùng `int main(void)` và khai báo đúng các hàm.

Câu 10 · Ba lượt visitor++ đồng thời

Trong mô hình đọc-cộng-ghi riêng rẽ, kết quả có thể là **1, 2 hoặc 3**. Nếu cả ba đọc 0 trước khi ghi, tất cả ghi 1. Nếu hai lượt ghi 1 rồi lượt còn lại đọc 1 và ghi 2, kết quả 2. Nếu ba phép tăng thực hiện tuần tự, kết quả 3. Bảo vệ bằng mutex hoặc phép tăng atomic cho kết quả 3 khi cả ba lượt hoàn tất. Với biến thường trong chương trình C/C++ đa luồng thật, data race không đồng bộ không có bảo đảm chỉ rơi vào ba giá trị này.

Đề 2022-2023 · CSC10007

Câu 1 · So sánh OS trên máy thật và máy ảo

Máy thật cho OS kiểm soát thiết bị vật lý trực tiếp qua driver, thường có ít lớp trung gian và dùng đầy đủ khả năng phần cứng được hỗ trợ. Máy ảo dùng phần cứng ảo, chia sẻ tài nguyên và có thể thêm chi phí xử lý I/O hoặc quản lý bộ nhớ. Máy ảo cho phép chạy nhiều môi trường, cô lập thử nghiệm, tạo snapshot và di chuyển môi trường khi nền tảng hỗ trợ. Hiệu năng thực tế tùy workload, hypervisor và hỗ trợ ảo hóa.

Câu 2 · Tìm kích thước vùng hệ thống từ tổng sector N

Đề cho 8 sector đầu, hai bảng bằng nhau, mỗi bảng có độ dài là bội số 8 sector. Một mục 32 bit = 4 byte theo dõi một cluster. Mỗi cluster dữ liệu có 8 sector. Dùng giả thiết **512 B/sector** theo mô hình bài giảng vì ảnh đề chưa nêu số byte/sector. Chọn kích thước bảng nhỏ nhất đủ chứa, và theo mô hình đề dùng một mục cho mỗi cluster dữ liệu.

Đặt k là số nhóm 8 sector của **mỗi** bảng. Vùng hệ thống chiếm 8 + 16k sector. Đặt M = floor((N − 8)/8). Vùng dữ liệu còn M − 2k cluster đầy đủ. Mỗi bảng chứa (8k × 512)/4 = 1.024k entry. Điều kiện đủ chỗ là 1.024k ≥ M − 2k, tương đương 1.026k ≥ M. Do đó **k = ceil(M/1.026)** và kích thước vùng hệ thống là **8 + 16k sector**. Nếu quy ước thêm hai entry dành riêng như FAT thật, thay M bằng M + 2 trong tử số. Đề mô tả bảng giản lược nên lời giải chính bám theo một entry mỗi cluster.

Với MSSV ví dụ N = 20.123.456, M = 2.515.431, k = 2.452. Vùng hệ thống chiếm **39.240 sector**. Dữ liệu còn 2.510.527 cluster. Một bảng chứa 2.510.848 entry. Nếu k = 2.451 thì sức chứa chỉ 2.509.824 trong khi số cluster dữ liệu là 2.510.529, nên thiếu. Nếu đề dùng B byte/sector, mẫu số 1.026 tổng quát thành 2B + 2.

Câu 3 · Mã hóa các entry của HeDieuHanh.pdf

Giả sử tạo alias 8.3 `HEDIEU~1.PDF` vì không có va chạm, thuộc tính theo đề là read-only (0x01), và thời điểm tạo có phần dưới giây bằng 0. Tên dài có 14 ký tự, nên cần hai LFN entry và một SFN entry. Tách 13 ký tự đầu `HeDieuHanh.pd` và mảnh cuối `f`. Trên đĩa ghi mảnh cuối trước, ordinal 0x42. Checksum của 11 byte `HEDIEU~1PDF` là **0x51**.

Trường	Giá trị	Byte lưu
Giờ tạo 01:23:45	0x0AF6, cộng phần lẻ 1 giây	0D là 64, 0E-0F là F6 0A
Ngày tạo 02/02/2020	0x5042	42 50
Ngày truy cập 01/11/2022	0x5561	61 55
Giờ sửa 01:23:45	0x0AF6, lưu theo bước 2 giây	F6 0A
Ngày sửa 02/01/2021	0x5222	22 52
Cluster 2.022	0x07E6	phần cao 00 00, thấp E6 07
Kích thước 12.345 B	0x00003039	39 30 00 00

Ba entry sau được in hai dòng cho mỗi entry. Nối hai dòng liên tiếp để được đúng 32 byte.

```
LFN #2   42 66 00 00 00 FF FF FF FF FF FF 0F 00 51 FF FF
         FF FF FF FF FF FF FF FF FF FF 00 00 FF FF FF FF
LFN #1   01 48 00 65 00 44 00 69 00 65 00 0F 00 51 75 00
         48 00 61 00 6E 00 68 00 2E 00 00 00 70 00 64 00
SFN      48 45 44 49 45 55 7E 31 50 44 46 01 00 64 F6 0A
         42 50 61 55 00 00 F6 0A 22 52 E6 07 39 30 00 00
```

Giờ sửa được lưu là 01:23:44 vì SFN không có byte phần lẻ riêng cho giờ sửa. Nếu môi trường sinh alias khác thì 11 byte SFN và checksum LFN thay đổi theo alias. Một số trình điều khiển đặt thêm bit archive khi tạo hoặc sửa. Bảng trên mã hóa đúng tập thuộc tính read-only mà đề cho.

Nguồn đối chiếu · CNTN_GK_HĐH_2025_2026.jpg, 22TN.jpg, GK_HĐH_2022_2023.jpg. Microsoft FAT Specification, mục 6-7.

09 · Thực hành FAT và tra nhanh

1. Giải FAT hw 1-2

Các sector và offset dưới đây tính từ đầu volume. Dữ liệu cho dưới dạng ảnh hex. Khi đọc, tách từng trường theo offset ở chương 4, đổi little-endian rồi kiểm tra chéo bằng offset xuất hiện trong ảnh.

Thông số	Bài 1	Bài 2
Loại FAT	FAT16	FAT16
Byte/sector	512	512
Sector/cluster	2	2
Số sector reserved	8	1
Số bảng FAT	2	2
Số sector mỗi FAT	32	40
Số entry RDET	512	512
Số sector RDET	32	32
Tổng số sector	16.352	20.448
Sector đầu FAT1	8	1
Sector đầu FAT2	40	41
Sector đầu RDET	72	81
Sector đầu Data	104	113
Cluster dữ liệu đầy đủ	8.124	10.167

Bài 1 · Bố cục và cây thư mục

Ở boot sector, offset 0x13 chứa `E0 3F`, tức 0x3FE0 = 16.352 sector. Offset 0x16 chứa `20 00`, tức 32 sector mỗi FAT. RDET bắt đầu tại 8 + 2 × 32 = 72, offset 0x9000. Data ở sector 104. Cluster 6 bắt đầu ở 104 + (6 − 2) × 2 = 112, offset 0xE000, đúng với ảnh.

```
/
|-- WINHEX.CNT       21 -> 22 -> EOC
|-- TIMEZONE.DAT      3 -> 4 -> 5 -> EOC
`-- XYZ/              6 -> EOC
    |-- ERROR.LOG     7 -> 8 -> EOC
    `-- EBCDIC.DAT    9 -> EOC
```

Mục WINHEX.CNT có byte cluster đầu `15 00`, tức 21. Đọc FAT[21] ở offset 2 × 21 = 42 byte tính từ đầu FAT1, thấy `16 00`, tức 22. FAT[22] là FFFF. Kích thước 0x0671 = 1.649 B vừa hai cluster 1.024 B. Mục XYZ có thuộc tính 0x10 nên đọc các entry của nó trong cluster 6. Hai entry `.` và `..` là liên kết thư mục, không phải file con độc lập. Trong ảnh RDET bài 1 có hàng toàn 0 trước các mục được minh họa. Khi giải, coi các mục hiện rõ là dữ liệu giả lập mà đề muốn đọc. Một trình đọc FAT theo chuẩn dừng khi gặp entry bắt đầu bằng 0x00.

Bài 2 · Bố cục và cây thư mục

Trường tổng sector 16 bit bằng 0. Đọc offset 0x20 được `E0 4F 00 00` = 20.448. FAT size là `28 00` = 40. RDET bắt đầu ở 1 + 2 × 40 = 81, tức offset 0xA200. Data ở 113. Sector 127 là sector đầu cluster 9 vì 113 + (9 − 2) × 2 = 127. Do đó các mục ở sector 127 là dữ liệu thư mục TMC.

```
/
|-- SAMPLE.WHS              2 -> 3 -> 4 -> EOC
|-- Boot Sector NTFS.tpl    5 -> 6 -> EOC
|   alias BOOTSE~1.TPL
|-- INODE.TPL               7 -> 8 -> EOC
`-- TMC/                    9 -> EOC
    |-- WINHEX.CNT         11 -> 12 -> EOC
    `-- ERROR.LOG          13 -> 14 -> EOC
```

Hai entry LFN trước alias BOOTSE~1.TPL có ordinal 0x42 và 0x01. Đọc theo thứ tự logic 1 rồi 2 để ghép “Boot Sector N” và “TFS.tpl”.

Thao tác trên Bài 2	FAT1	FAT2	RDET	Data
Move ERROR.LOG từ TMC ra root	Không đổi	Không đổi	Sector 81	Sector 127
Delete TMC/WINHEX.CNT	Sector 1	Sector 41	Không đổi	Sector 127

Move dùng entry trống trong sector 81 và đánh dấu entry cũ của ERROR.LOG tại 0xFE60 trong sector 127. Chuỗi 13 → 14 giữ nguyên. Delete đánh dấu entry tại 0xFE40 và đặt FAT[11], FAT[12] thành 0 trong hai bảng. Mỗi entry FAT16 dài 2 B nên offset 22 và 24 vẫn thuộc sector đầu của mỗi bảng. Hai thao tác xét cập nhật logic bắt buộc, bỏ qua ghi timestamp tùy driver. Kết quả delete giống nhau dù thực hiện sau move hay xét riêng.

2. Tra nhanh khi làm bài

Dạng bài	Các bước nên ghi trên giấy
So sánh polling và interrupt	Ai phát hiện sự kiện, sự kiện thưa hay dày, overhead, độ trễ.
So sánh PIO và DMA	Ai chuyển dữ liệu, chi phí thiết lập, chi phí theo lượng dữ liệu, đại lượng đang tối ưu.
Lập lịch đĩa	Vị trí đầu, biên, hướng, thứ tự đi, trị tuyệt đối từng chặng.
Đọc FAT	BPS, SPC, reserved, số FAT, FAT size, RootEntCnt, TotalSectors, FirstData, chuỗi cluster.
Thao tác FAT	Đánh dấu entry ở vùng nào, có đổi chuỗi không, có ghi nội dung mới không.
Điều phối CPU	Sự kiện đến, ready queue, Gantt, F và C, R = F − A, T = C − A.
fork() và printf	Vẽ nhánh cha/con, đếm lệnh chạy, xem bộ đệm và kiểu thoát.
Race condition	Tách đọc-sửa-ghi, viết thứ tự xen kẽ, nêu biến dùng chung và cách bảo vệ.
Dịch địa chỉ	Chia lấy page và offset, tra frame, nhân cỡ trang rồi cộng offset.

3. Tự kiểm tra trước khi làm đề

- Mô tả được một lần đọc tập tin đi từ chương trình đến kernel và thiết bị, rồi quay lại chương trình.
- Tách được lập lịch CPU khỏi lập lịch đầu đọc, dù cả hai có thuật toán tên FCFS.
- Tính được FirstData từ BPB và dùng cluster 2 làm mốc, không nhầm offset hex với số sector.
- Đọc đúng thứ tự trên đĩa của LFN, tính được SFN và kiểm tra đủ 32 byte mỗi entry.
- Biết mốc lần đầu chạy, mốc hoàn tất và các khoảng ở ready queue trong biểu đồ Gantt.
- Giải thích được điều kiện của từng đáp án khi đề cũ để thiếu dữ kiện.

Tài liệu gốc

- Syllabus-Introduction to Operating Systems.pdf, trang 3-5. 1 Ngay 1 intro.pdf, slide 7-8 về kế hoạch dự kiến.
- 2 Architecture_Tour.pdf, 3 disk 1.pdf, 4a File system.pdf, 4b Implement File System.pdf, 5 processes.pdf, 6 Dieu phoi CPU.pdf, 6 Process scheduling.pdf và 7 synchronization.pdf.
- SSD.pptx, ssd_continue.pptx, 8 Memory management.ppt và 8 Virtual memory.ppt.
- Hang out.pdf, FAT hw 1-2.pdf và ảnh đề GK_HĐH_2022_2023.jpg, 22TN.jpg, CNTN_GK_HĐH_2025_2026.jpg.
- Silberschatz, Galvin và Gagne. Operating System Concepts, ấn bản 10. Các chương liên quan được ghi ở cuối từng chương.
- Microsoft FAT Specification, mục 3-7 về bố cục, entry và LFN.